

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 07 • NIVEL 2 - INTERMEDIO

Clase 07: Grafos, Matrices de Adyacencia y Recorridos BFS/DFS

«Grafos como Redes de Ciudades y Rutas de Vuelo»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 07** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender vértices, aristas, listas de adyacencia y algoritmos de búsqueda en grafos.

Competencia Práctica: Implementar Búsqueda en Amplitud (BFS con colas) y Búsqueda en Profundidad (DFS con pilas/recursión).

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 07: Grafos, Matrices de Adyacencia y Recorridos BFS/DFS

Los grafos modelan relaciones complejas de red entre entidades (redes sociales, mapas, dependencias).

☀ Metáfora Central: Grafos como Redes de Ciudades y Rutas de Vuelo

Un grafo es un mapa de aeropuertos (nodos) conectados por vuelos (aristas).

Principios Teóricos y Modelo Mental

BFS (Breadth-First Search) explora por capas concéntricas usando una Cola FIFO; encuentra el camino más corto.

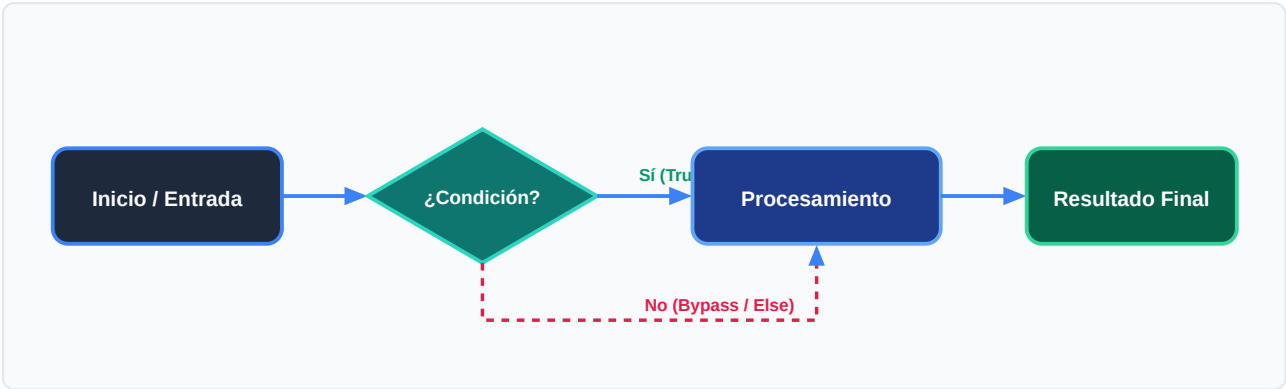
DFS (Depth-First Search) explora hasta el fondo de cada rama usando una Pila o recursión.

⚡ Regla de Oro en Python

En grafos con ciclos, mantén siempre un conjunto `'visitados = set()'` para evitar bucles infinitos.

Arquitectura y Movimiento de Datos en Memoria

Exploración por niveles con BFS vs exploración profunda con DFS.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Representación del grafo como lista de adyacencia (dict).	Grafo en memoria.
2. Evaluación	Inicialización de cola FIFO con el nodo de origen.	Cola = [inicio], visitados = {inicio}.
3. Transformación	Extracción del nodo y visita a vecinos no explorados.	Vecinos encolados.
4. Retorno / Salida	Fin de exploración tras vaciar la cola.	Árbol de expansión BFS obtenido.



Visualización Mental

BFS es como una onda en el agua que se expande en círculos; DFS es como entrar a un laberinto siempre hacia adelante.

Código de Demostración en Python 3.10+

Algoritmo BFS para encontrar camino más corto:

main.py (Python 3.10+)

PEP 8 Compliant

```
from collections import deque

grafo = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}

def bfs(grafo: dict, inicio: str) -> list[str]:
    visitados = {inicio}
    cola = deque([inicio])
    recorrido = []
    while cola:
        nodo = cola.popleft()
        recorrido.append(nodo)
        for vecino in grafo.get(nodo, []):
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
    return recorrido

print("Recorrido BFS:", bfs(grafo, "A"))
```

Análisis de Ingeniería del Código

Uso de collections.deque y conjunto visitados para garantizar tiempo $O(V + E)$.

💡 Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Ciclos infinitos en grafos.

⚠ Gotcha Frecuente (Trampa de Principiante)

No registrar los nodos en visitados provoca un RecursionError o bucle infinito.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
def dfs(nodo):  
    for v in grafo[nodo]: dfs(v) # ❌ Sin  
    control de visitados en grafo cíclico
```

✅ Patrón Correcto

```
def dfs(nodo, visitados=None):  
    if visitados is None: visitados = set()  
    visitados.add(nodo)  
    for v in grafo[nodo]:  
        if v not in visitados: dfs(v, visitados)  
# ✅ Seguro
```

🛡 Consejo de Resiliencia en Producción

Para grafos con pesos en las aristas, usa el algoritmo de Dijkstra con heapq.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 07: Grafos, Matrices de Adyacencia y Recorridos BFS/DFS**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Grafos como Redes de Ciudades y Rutas de Vuelo
2. Regla de Oro	En grafos con ciclos, mantén siempre un conjunto 'visitados = set()' para evitar bucles infinitos.
3. Gotcha Crítico	No registrar los nodos en visitados provoca un RecursionError o bucle infinito.
4. Buenas Prácticas	Para grafos con pesos en las aristas, usa el algoritmo de Dijkstra con heapq.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Implementa una función que determine si existe un camino entre dos nodos dados en un grafo.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.